Morphix – Architekturdokument (Entwurf)

Nov 11, 2026 · @Jule Rotteveel

Das Architekturdokument beschreibt, wie Morphix intern aufgebaut ist, um die Anforderungen aus dem Anforderungsdokument und das nach außen sichtbare Verhalten aus dem Spezifikationsdokument umzusetzen. Es umfasst Technologieübersicht, Architekturübersicht, Struktur (UML-Klassendiagramm) und Verhalten (UML-Sequenzdiagramm).

1 Technologieübersicht

Die Technologiewahl folgt drei Kriterien: Die Anwendung muss die Windows-Schnittstellen (Dienste, Ereignisanzeige, Aufgabenplanung) direkt nutzen können, auf Windows 10/11 ohne Zusatzinstallationen laufen (NFA09) und mit den vorhandenen Grundkenntnissen in C# im Zeitplan umsetzbar sein (Risiko R2).

1.1 Plattform und Werkzeuge

Technologie	Version	Zweck	Begründung	Verworfene Alternative
C#	14	Programmiersprache	Moderne objektorientierte Hochsprache, direkter Zugriff auf Windows-APIs, Vorkenntnisse vorhanden	Python (Windows-APIs nur über Zusatzpakete, kein natives Desktop-UI-Framework)
.NET	10 (LTS)	Laufzeitumgebung	Aktuelle LTS-Version mit Support bis November 2028; .NET 8 hat am 10.11.2026 das Supportende erreicht	.NET 8 (kein Support mehr), .NET Framework 4.8 (nicht weiterentwickelt)
WPF	Teil von .NET 10	Grafische Oberfläche	Ausgereiftes Desktop-Framework mit Datenbindung, die MVVM unmittelbar unterstützt	WinUI 3 (aufwendigere Auslieferung), Windows Forms (keine saubere Trennung von Oberfläche und Logik)
Visual Studio	2026	Entwicklungsumgebung	Unterstützt .NET 10, XAML-Designer, Debugger, Testexplorer	JetBrains Rider (kostenpflichtig)

Technologie	Version	Zweck	Begründung	Verworfene Alternative
Git / GitHub	–	Versionsverwaltung	Vorgabe der Aufgabenstellung; tägliche Commits als Maßnahme gegen R8	–
draw.io	–	UML-Modellierung	Von der IU empfohlen, importiert Mermaid-Quelltext	Visio (kostenpflichtig)

1.2 Bibliotheken (Third Party)

Die Aufgabenstellung verlangt, Fremdbibliotheken explizit zu dokumentieren und zu begründen. Alle Pakete werden über NuGet eingebunden und stehen unter freien Lizenzen; fremder Anwendungscode wird nicht übernommen.

Paket	Lizenz	Einsatz	Begründung
CommunityToolkit.Mvvm	MIT	Basisklassen und Befehle für ViewModels	Erzeugt Boilerplate für Eigenschaftsänderungen und Commands per Quellcodegenerator; schlanker als Prism
Microsoft.Extensions.DependencyInjection	MIT	Dependency Injection	Standardcontainer von .NET; ermöglicht den Austausch echter und simulierter Datenquellen (Demo-Modus)
System.ServiceProcess.ServiceController	MIT	Windows-Dienste abfragen und steuern	Offizielle Microsoft-Schnittstelle zum Service Control Manager
System.Diagnostics.EventLog	MIT	Ereignisanzeige lesen (Should, US06)	Offizielle Microsoft-Schnittstelle zum Windows Event Log
TaskScheduler	MIT	Aufgabenplanung lesen (Should, US10)	Verwaltete Hülle um die COM-Schnittstelle des Task Schedulers; direkte COM-Interop wäre fehleranfällig, das Auswerten von schtasks.exe sprachabhängig
OxyPlot.Wpf	MIT	Diagramme (Could, US12)	Leichtgewichtig, nur bei Umsetzung von US12 eingebunden
xUnit, NSubstitute, coverlet.collector	Apache 2.0 / BSD-3 /	Unit-Tests, Test-Doubles,	Verbreiteter .NET-Teststack; Coverlet misst die in NFA10 geforderte Abdeckung

Paket	Lizenz	Einsatz	Begründung
	MIT	Testabdeckung	

Bordmittel von .NET ohne Zusatzpaket: System.Text.Json (Systemprofile), System.IO.Compression (ZIP-Export), System.Net.WebUtility (HTML-Kodierung im Bericht), System.Security.Principal (Prüfung der Administratorrechte).

2 Architekturübersicht

2.1 Gewählte Architektur

Morphix folgt einer **Schichtenarchitektur mit vier Schichten**, kombiniert mit dem Muster **Model-View-ViewModel (MVVM)** in der Präsentationsschicht und dem Prinzip **Ports and Adapters** für alle Zugriffe auf Windows (Cockburn 2005; Martin 2017).

Schicht	Projekt	Inhalt	Darf zugreifen auf
Präsentation	Morphix.App	XAML-Views, ViewModels, Start und Konfiguration der Dependency Injection	Anwendung
Anwendung	Morphix.Core	Services, die die Geschäftsprozesse P1–P7 steuern	Domäne, Schnittstellen
Domäne	Morphix.Core	Geschäftsobjekte aus dem Datenmodell, Geschäftsregeln GR01–GR10, Algorithmen (Parsen, Gruppieren, Maskieren) sowie die Schnittstellen (Ports) zu externen Systemen	nichts
Infrastruktur	Morphix.Infrastructure	Adapter für Dateisystem, Ereignisanzeige, Service Control Manager, Aufgabenplanung, JSON und HTML/ZIP sowie simulierte Demo-Adapter	Schnittstellen der Domäne
Tests	Morphix.Tests	Unit- und Integrationstests	alle

Abhängigkeiten zeigen nur nach innen: Die Infrastruktur implementiert Schnittstellen, die in Morphix.Core definiert sind, nicht umgekehrt (Dependency-Inversion-Prinzip). Morphix.Core zielt auf `net10.0` ohne Windows-Bezug und lässt sich deshalb vollständig ohne echte Dienste oder Logdateien testen.

Abbildung: Schichtenübersicht (Datei *schichten.png*).

2.2 Begründung

Entscheidung	Begründung	Bezug
Schichtenarchitektur	Klare Verantwortlichkeiten, einfach nachvollziehbar, für eine lokale Einzelplatzanwendung angemessen (Sommerville 2020)	NFA10
MVVM	Trennt Oberfläche und Logik; ViewModels sind ohne Oberfläche testbar; von Microsoft für WPF empfohlen	NFA10, NFA03
Ports and Adapters	Echte und simulierte Datenquellen implementieren dieselbe Schnittstelle; der Demo-Modus ist dadurch kein Sonderfall im Code, sondern ein Austausch der Adapter	US02, GR04, R3
Asynchrone Verarbeitung (async/await, IProgress)	Sammeln und Analysieren blockieren die Oberfläche nicht; Fortschritt wird gemeldet	NFA02
Rechteprüfung im Service statt nur in der Oberfläche	Die Regel GR03 gilt auch dann, wenn eine Schaltfläche fälschlich aktiv wäre	NFA05

2.3 Verworfene Alternativen

Alternative	Grund für die Ablehnung
Logik im Code-Behind der Fenster	Nicht testbar, Demo-Modus nur über viele Fallunterscheidungen möglich
Vollständige Clean Architecture mit getrennten Projekten je Ring	Mehr Projekte und Abbildungscode als für den Umfang nötig; widerspricht dem knappen Zeitplan (R1)
Client-Server oder Microservices	Morphix arbeitet laut NFA06 bewusst ohne Netzwerk; ein Server brächte nur Komplexität

2.4 Eingesetzte Entwurfsmuster

Muster	Einsatz in Morphix
MVVM / Beobachter	ViewModels melden Änderungen über INotifyPropertyChanged, die Views aktualisieren sich per Datenbindung
Adapter	Windows-Klassen wie ServiceController werden hinter IDienstGateway gekapselt
Strategie	Echte oder Demo-Datenquellen werden je Profil gewählt

Muster	Einsatz in Morphix
Fabrik	IDatenquellenFabrik liefert passend zum Profil (istDemo) die Datenquellen
Repository	IProfilRepository kapselt das Laden und Speichern der JSON-Profile
Dependency Injection	Alle Abhängigkeiten werden beim Start in App.xaml.cs registriert

3 Struktur

Das UML-Klassendiagramm (Datei *klassen.png*) zeigt die Hauptkomponenten je Schicht mit ihren wichtigsten Attributen, Methoden und Abhängigkeiten. Die Geschäftsobjekte (Systemprofil, Diagnose, Logeintrag usw.) sind bereits im Klassendiagramm des Spezifikationsdokuments beschrieben und werden hier nur als Typen referenziert.

Klasse / Schnittstelle	Schicht	Verantwortung
MainViewModel	Präsentation	Profil und Zeitraum wählen, Analyse starten, Navigation
DiensteViewModel	Präsentation	Dienststatus alle 5 s aktualisieren, Dienstaktionen mit Bestätigung
BerichtViewModel	Präsentation	Vorschau, Option „Maskieren“, Export
DiagnoseService	Anwendung	Steuert P2, P3, P6 und das Erfassen für P7; erzeugt eine abgeschlossene Diagnose
DienstService	Anwendung	P4/P5; prüft GR03 (Profil, Rechte, Bestätigung) und GR04 (Demo)
BerichtService	Anwendung	P7; erzeugt den Bericht (GR09) und löst den Export aus
ProfilService	Anwendung	P1; lädt, validiert (GR02) und speichert Profile
LogParser	Domäne	Zerlegt Zeilen in Logeinträge; mehrzeilige Meldungen nach GR06
FehlerGruppierer	Domäne	Normalisiert Meldungen und bildet Fehlergruppen (P3, GR07)
DatenMaskierer	Domäne	Ersetzt Benutzernamen und IP-Adressen (US15, GR10)
ZeitraumRegel	Domäne	Prüft GR01
IDatenquellenFabrik	Schnittstelle	Liefert echte oder simulierte Datenquellen
ILogLeser, IDienstGateway,	Schnittstelle	Zugriff auf Logdateien, Dienste und Aufgaben

Klasse / Schnittstelle	Schicht	Verantwortung
IAufgabenGateway		
IProfilRepository, IBerichtExporter, IRechtePruefer	Schnittstelle	Profile speichern, Bericht exportieren, Adminrechte prüfen
Windows… / Demo… / Json… / HtmlZipExporter	Infrastruktur	Konkrete Implementierungen der Schnittstellen

4 Verhalten

Das UML-Sequenzdiagramm (Datei *sequenz.png*) zeigt den Kernprozess P7 „Support-Bericht erstellen“, der bereits im Spezifikationsdokument als Aktivitätsdiagramm beschrieben ist. Es verdeutlicht drei Architekturentscheidungen:

- Die Fabrik entscheidet anhand des Profils, ob echte oder Demo-Datenquellen verwendet werden; der DiagnoseService kennt den Unterschied nicht.
- Logs, Dienststatus und Aufgaben werden parallel erfasst (`par`-Fragment), wie in P7 spezifiziert; der Aufgaben-Zweig gehört zur Should-Anforderung US10.
- Die Maskierung läuft im BerichtService vor dem Export, sodass ungeschützte Daten den Rechner nicht verlassen (GR10, NFA06).

5 Querschnittliche Aspekte

Aspekt	Lösung	Bezug
Fehlerbehandlung	Leser fangen Ein-/Ausgabefehler je Datei ab, überspringen die Datei und sammeln eine Meldung; die Diagnose enthält eine Liste übersprungener Dateien	NFA07
Nebenläufigkeit	Einlesen und Gruppieren per Task.Run, parallele Erfassung per Task.WhenAll; Statusabfrage der Dienste per PeriodicTimer alle 5 s	NFA02, P4
Lesender Dateizugriff	Dateien mit FileAccess.Read und FileShare.ReadWrite öffnen, damit auch Logs gelesen werden können, in die das System gerade schreibt	GR05, NFA08
Sicherheit im Bericht	Alle Inhalte aus Logs werden vor dem Einfügen ins HTML kodiert, damit eingebettete Skripte nicht ausgeführt werden (Cross-Site-Scripting, OWASP)	NFA06

Aspekt	Lösung	Bezug
Speicherorte	Profile unter %AppData%\Morphix\Profile; Demo-Profil und Demo-Daten im Programmordner	NFA10
Herstellerunabhängiges Parsen	Jede Logquelle kann ein Zeilenmuster (regulärer Ausdruck mit den Gruppen *zeit*, *stufe*, *text*) festlegen; ohne Angabe gilt ein Standardmuster	Abgrenzung, NFA10

6 Auswirkungen auf die Dokumente aus Phase 1

Zwei Entscheidungen dieses Dokuments ändern Phase-1-Inhalte und gehören ins Änderungsprotokoll der Projektdokumentation:

Nr.	Dokument / Abschnitt	Änderung	Begründung
Ä8	Projektübersicht, NFA09	.NET 8 → .NET 10 (LTS)	Supportende von .NET 8 am 10.11.2026; die Aufgabenstellung setzt die jeweils neueste Laufzeitumgebung voraus
Ä9	Spezifikation – Datenmodell, Logquelle	Neues optionales Attribut *zeilenmuster*	Logformate verschiedener Hersteller lassen sich nur so ohne Codeänderung einlesen (NFA10)

7 Quellen

- Cockburn, A. (2005). *Hexagonal Architecture.* https://alistair.cockburn.us/hexagonal-architecture/
- Gamma, E., Helm, R., Johnson, R. & Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley.
- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design.* Prentice Hall.
- Microsoft. (2026). *.NET and .NET Core Support Policy.* https://dotnet.microsoft.com/platform/support/policy/dotnet-core
- Microsoft. (2026). *.NET 8 and .NET 9 will reach End of Support on November 10, 2026.* https://devblogs.microsoft.com/dotnet/dotnet-8-9-end-of-support/
- Microsoft. (2025). *Model-View-ViewModel (MVVM).* https://learn.microsoft.com/dotnet/architecture/maui/mvvm
- Object Management Group. (2017). *Unified Modeling Language (UML) Version 2.5.1.* https://www.omg.org/spec/UML/2.5.1
- OWASP Foundation. (2021). *OWASP Top 10: A03 Injection.* https://owasp.org/Top10/
- Sommerville, I. (2020). *Software Engineering* (10. Aufl.). Pearson.

Hinweis: Abrufdaten der URLs beim Übertragen ergänzen.